

Seven defects found in ttrack-vis's provenance sidebar, patched in place

GO-Compass renders its undo/redo history through `ttrack-vis`, the visualization layer that ships alongside Ttrack. Working through its annotation, bookmarking, and graph rendering features surfaced seven defects in the library's own compiled source — none of them GO-Compass bugs. Each was traced to its root cause and corrected with a minimal, targeted change, distributed as a `patch-package` patch so the fixes survive every future `npm install`.

Patch file `@visdesignlab+ttrack-vis+2.0.0.patch` Mechanism `patch-package`

Diagnosed with Claude

7

Defects fixed

2

Compiled files patched

4

Components touched

✓

Verified via clean reinstall

THE DEFECTS

01 Annotation box silently discarded every keystroke

BackboneNode.tsx

OBSERVED Typing into a node's annotation textarea had no visible effect whatsoever — the box stayed empty no matter what was typed.

CAUSE The textarea's `onChange` handler was a hardcoded no-op — dead code shipped in the compiled bundle, with no way to reach it through any public prop.

```
- var handleInputChange = function() {};  
+ var handleInputChange = function(e) {  
+   setAnnotateText(e.target.value);  
+ };
```

02 Closing an unannotated node's box, then annotating it, crashed the app

BackboneNode.tsx

OBSERVED `TypeError: Cannot read properties of undefined (reading 'trim')` — reproducible only after closing a never-annotated node's box once first, which made the failure look intermittent.

CAUSE Closing the box reset the textarea's state from the node's last annotation with no fallback for "there isn't one yet" — leaving it as undefined instead of an empty string, which the next save then choked on.

```
- setAnnotateText(prov.getLatestAnnotation(id)?.annotation);  
+ setAnnotateText(prov.getLatestAnnotation(id)?.annotation ?? '');
```

03 Bookmarking two nodes with the same label hid one of them

BookmarkListView.tsx

OBSERVED Bookmarks would inconsistently vanish from the "Bookmarks/Annotations" tab — not every bookmarked node made it into the list.

CAUSE The list identified each row by the node's *label text* rather than its unique id. Two nodes sharing a label — easy, since action labels are short and reused — collapsed into a single rendered row.

```
- keyAccessor: (d) => d.label,  
+ keyAccessor: (d) => d.id,
```

04 Bookmark entries showed a pointer cursor but did nothing when clicked

BookmarkListView.tsx · ProvVis.tsx

OBSERVED Clicking a node in the main graph jumps history to that point, as expected. Clicking the same node's row in the Bookmarks tab — styled with the identical pointer cursor — was inert.

CAUSE The click→navigate callback was wired for the main graph's nodes but never threaded through to the bookmark list, which had no way to receive it.

```
<g transform={translate(x, y)}  
+   onClick={() => changeCurrent(d.id)}  
>
```

05 Annotating a node without bookmarking it left no trace anywhere

BookmarkListView.tsx

OBSERVED The tab is named **"Bookmarks/Annotations"** — a label ttrack-vis hardcodes itself — yet a node you annotated without also bookmarking never appeared in it.

CAUSE The list's filter checked only whether a node was bookmarked; it never asked whether a node carried an annotation at all.

```
- if (node.bookmarked) { list.push(node); }  
+ if (node.bookmarked || isAnnotated) { list.push(node); }
```

06 Editing an existing annotation never changed what was shown on screen

BackboneNode.tsx · BookmarkNode.tsx

OBSERVED The save itself worked — but the annotation preview beside a node stayed frozen on whatever was typed the very first time, no matter how many times it was edited afterward.

CAUSE Ttrack correctly appends each edit to the end of a node's annotation history, but the preview display read only the *first* entry ever recorded, not the latest one — the two coincide until a node's second edit, which is exactly why the bug went unnoticed at first.

```
- annotations[0].annotation  
+ annotations[annotations.length - 1].annotation
```

07 Long provenance graphs outgrew their own scroll area

ProvVis.tsx

OBSERVED In sessions with a long action history, the sidebar's scrollbar reached its limit well before the bottom of the graph — some nodes stayed permanently out of view, no matter how far down it was possible to scroll.

CAUSE The SVG's height already grows past its initial measured size once the graph gets deep enough — but the growth calculation used each node's raw depth number directly as a pixel count, instead of multiplying it by the actual per-node vertical spacing. A graph 700 actions deep set the SVG's height to 700 pixels instead of the $700 \times 30 = 21,000$ pixels its content actually needed.

```
- height: maxHeight < height ? height : maxHeight,  
+ height: maxHeight * yOffset < height ? height : maxHeight * yOffset,
```

HOW THE FIX IS DELIVERED

All seven changes live directly in ttrack-vis's compiled output (`ttrackvis.esm.js` and `ttrackvis.cjs.development.js`), captured as a single `patch-package` diff at `patches/@visdesignlab+ttrack-vis+2.0.0.patch` . A `postinstall` script reapplies it automatically after every `npm install` — on a fresh clone, in CI, or on a teammate's machine — with no manual step and no fork to maintain.

- ✓ Every fix confirmed against the library's own source before being written, not assumed from symptoms.
- ✓ Each patched file reinstalled from a clean state and re-grepped afterward to confirm the fix — not just the tool's exit code — repeated after every additional fix.
- ✓ The committed patch diffed against a fresh regeneration from live `node_modules` : byte-identical, confirming no drift.
- ✓ Confirmed working end-to-end in a running copy of GO-Compass.

GO-Compass · provenance & undo-redo tooling · patch notes compiled with Claude